

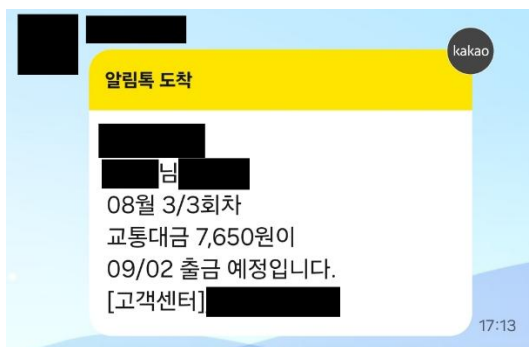
대중교통 사용 내역 관리 앱 개발

제목은 적절한 자신의 요구에 따라 변경

1. 사업 개요

- 사업명 (프로젝트명): 대중교통 사용 내역 관리
- 발주자 (작성자): 정유운
- 개발 기한: 2026년 9월 9일 ~ 2026년 9월 28일
- 추진 배경 및 목적:

대중교통을 자주 사용하면 내가 생각한 돈 보다 더 많이 나가는 것을 볼 수 있다. 그래서 해당 대중교통을 사용할 때 후불 교통 카드의 경우 내가 어디에서 버스나 지하철 요금을 지불하였는지 세부적으로 알 수 없어 추후 대중교통 사용을 관리하기에 어려움이 따른다. 따라서 사용 내역에 따라 대중교통을 효율적으로 사용하기 위해 추진



(Image 1) 총합 교통비

2. 개발 대상 및 환경 (제약 사항)

- 대상 플랫폼: [] CLI (콘솔 화면) / [] GUI (화면 창) / [●] 웹 / [] 기타
- 사용 프로그래밍 언어: JavaScript (+ Html, CSS)
- 데이터 저장 방식: [] Text / CSV / [●] JSON / [~~데이터베이스(SQLite 등)~~]
- 기타 개발 제약조건: 외부 라이브러리 사용 X

3. 상세 요구사항

3.1 기능 요구사항 (Functional Requirements)

요구사항 ID	기능명	상세 내용 및 처리 로직	우선순위
FR-01	요금 데이터 지정	코드 내부에 자체적으로 더미 데이터식으로 지역 별, 대중교통 별 요금을 저장	상
FR-02	노선 템플릿 등록	로컬 스토리지를 사용하여 노선 템플릿 등록 및 저장 ----- 주로 대중교통을 사용한 위치 및 해당 요금과 환승 가능 여부, 탑승 시간 (+ 출발지부터 목적지 까지의 노선) -----	상
FR-03	대중교통 사용 내역 등록	로컬 스토리지를 사용하여 노선 템플릿 혹은 임의 생성을 통한 대중교통 사용 내역 등록 및 저장 ----- 주로 탑승한 대중교통 및 시간, 환승 여부를 저장하고 만약 노선 템플릿에 의해 등록된 경우 1개씩 저장되는 것이 아니고 여러 개가 저장 될 수 있음 (만약 템플릿에서 탑승하는 대중 교통이 3개면 3개가 저장됨) -----	상
FR-04	화면 구현	FR-01 ~ 03 에 의해 저장된 데이터를 이용하여 템플릿, 내역 생성 버튼이나 내역이 저장된 화면을 보여준다 -----	상

		이벤트를 활용하여 사용자와의 상호작용을 통해 가시성 + -----	
FR-05	지불 금액 계산	FR-04에서 구현된 화면 아래 1개월 혹은 1년 단위의 지불 금액을 계산한다 ----- 추가적으로 특정 내역만을 골라 그것에 대한 단위 지불 금액 계산도 가능하게끔 했으면 좋겠다. -----	중
FR-06	FR-02, 03 수정 및 삭제	노선 템플릿, 사용 내역 데이터를 잘못 등록할 경우 해당 데이터의 수정 및 삭제가 필요하다	하

3.2 데이터 요구사항 (Data Requirements)

- **영속성(Persistence):** 브라우저 내 로컬 스토리지를 사용하여 사용자가 탭을 닫아도 데이터가 삭제되지 않게 함
- **관리할 데이터 구조:**

1. 요금 데이터 (JSON)

```
{
  "Region" : [
    {
      "Name": "Seoul",
      "Train": [
        {
          "Child" : 1000,
          "Youth" : 1200,
          "Adult" : 2000,
          "TransferDC" : 0
        }
      ],
      "Bus": [
        {
          "Child" : 800,
          "Youth" : 1000,
          "Adult" : 1500,
          "TransferDC" : 0.1
        }
      ]
    }
  ],
  "_Read Me" : "이런 방식을 사용하라는 것이지 데이터는 자율적으로 조정해도 됨"
}
```

2. 노선 템플릿 데이터 (JSON)

```
{
  "Template" : [
    {
      "Name" : "Test",

```

```

    "TP List" : [
      {
        "Name" : "대구 3호선 칠곡경대병원 방면",
        "Type" : "Train",
        "Region" : "Daegu",
        "AgeType" : "Abult",
        "IsTransfer" : false
      },
      {
        "Name" : "724",
        "Type" : "Bus",
        "Region" : "Daegu",
        "AgeType" : "Abult",
        "IsTransfer" : true
      }
    ]
  }
]
}

```

3. 대중교통 사용 내역

```

{
  "Log" : [
    {
      "Name" : "대구 3호선 칠곡경대병원 방면",
      "Time" : "2026-09-05T10:40:00+09:00",
      "Type" : "Train",
      "Region" : "Daegu",
      "AgeType" : "Adult",
      "IsTransfer" : false
    },
    {
      "_more" : "위랑 같은 형식으로 계속 추가하면 됨 "
    }
  ]
}

```

3.3 예외 및 예외 처리 요구사항 (Exception Handling)

- **EH-01 (잘못된 입력):** 화면 구현 시 사용자가 템플릿 혹은 내역을 추가할 때 옵션을 직접 입력하게 두지 말고 스크립트가 지정된 버튼 혹은 선택을 통해서 접근하여 입력 예외가 대부분 발생하지 않도록 한다. (대중교통의 이름을 입력하는 경우 문자열 앞 뒤에 공백이 생기지 않도록 해야합니다.)
- **EH-02 (데이터 오류 방지):** JSON을 파싱 및 쓰기 연산을 할 때 규격에 맞게 저장되어야 한다

3.4 비기능 요구사항 (Non-Functional Requirements)

- 개발자가 코드를 어떤식으로 짜든 상관은 없으나 처음 소스코드에 사용된 기능에 대한 설명을 반드시 첨부할 것

4. 프로젝트 관리

- 개발자는 필요시 발주자와의 연락을 통하여 프로그램 내 필요한 기능을 보강할 수 있도록 한다
- 개발자는 시간이 날 때 발주자와 함께 설계를 점검 할 수 있도록 한다

5. 최종 산출물 (Deliverables)

개발자는 기한 내에 다음 항목을 완료하여 제출해야 한다.

1. HTML, CSS, JavaScript가 포함된 프로젝트 폴더 (혹은 .zip): 최종 결과 프로그램(3개가 반드시 연동 되서 나오게 html 문서를 꾸민다)
2. 데이터 파일 : 로컬 스토리지에서 데이터를 파싱 해 내부에 있는 요소를 알 수 없으므로 해당 데이터의 규격이 맞춰진 JSON 파일을 첨부 할 수 있도록 한다 (요금 데이터, 템플릿 데이터, 내역 데이터)

6. 검수 및 평가 기준 (Acceptance Criteria)

- [] 필수 요구사항 (FR-01 ~ 04)이 제대로 구현되고 동작하는가?
- [] 로컬 스토리지 내 문자열 파싱 및 JSON 변환이 제대로 이루어지고 있는가?
- [] JSON-> 문자열로 변환된 데이터가 로컬 스토리지에 잘 저장되고 있는가?
- [] 프론트엔드 부분 (화면구현)이 정상적으로 작동하는가?
- [] 소스코드 설명 부분이 잘 작성 되었는가?